

Movie Streaming Database

Entity Relationships

Derived from the ER Diagram and Table Schema

Overview

The Movie Streaming Database has four core relationships connecting its entities. Two are pure many-to-many (M:N) relationships resolved into junction/bridge tables that carry their own attributes, one is a many-to-many relationship resolved into a simple composite-key bridge table, and one is a direct one-to-many (1:N) relationship implemented with a single foreign key.

1. USERS ↔ SUBSCRIPTION

Relationship name: has_subscription

Cardinality: 1 : N — one user can have many subscriptions; each subscription belongs to exactly one user

Participation: USERS side is (0,N) — optional, a user may or may not have a subscription. SUBSCRIPTION side is (1,1) — mandatory, every subscription row must belong to exactly one user.

Implemented via: subscription.user_id (FK referencing users.user_id)

2. USERS ↔ MOVIES (via WATCH_HISTORY)

Relationship name: watches

Cardinality: M : N — a user watches many movies; a movie is watched by many users

Participation: USERS side is (0,N) as *viewer*. MOVIES side is (0,N) as *watched*. Both optional.

Implemented via: watch_history table, with watch_history.user_id (FK → users) and watch_history.movie_id (FK → movies), plus its own attributes (watch_date, watched_minutes, completed).

3. USERS ↔ MOVIES (via RATINGS)

Relationship name: rates

Cardinality: M : N — a user rates many movies; a movie is rated by many users

Participation: USERS side is (0,N) as *rater*. MOVIES side is (0,N) as *rated*. Both optional.

Implemented via: ratings table, with ratings.user_id (FK → users) and ratings.movie_id (FK → movies), plus its own attributes (score, review).

4. MOVIES ↔ GENRES (via MOVIE_GENRE)

Relationship name: has_genre

Cardinality: M : N — a movie can belong to many genres; a genre applies to many movies

Participation: MOVIES side is (1,N) — mandatory, every movie must have at least one genre. GENRES side is (0,N) — optional, a genre may or may not have movies assigned to it.

Implemented via: movie_genre junction table, with movie_genre.movie_id (FK → movies) and movie_genre.genre_id (FK → genres) forming a composite primary key.

Summary Table

Relationship	Entities	Type	Resolved via
has_subscription	USERS ↔ SUBSCRIPTION	1 : N	Direct FK (subscription.user_id)
watches	USERS ↔ MOVIES	M : N	WATCH_HISTORY bridge table
rates	USERS ↔ MOVIES	M : N	RATINGS bridge table
has_genre	MOVIES ↔ GENRES	M : N	MOVIE_GENRE bridge table

Note

USERS and MOVIES actually share **two separate M:N relationships** (*watches* and *rates*) — this is why they are resolved into two distinct bridge tables (WATCH_HISTORY and RATINGS) rather than one. A single junction table cannot cleanly carry two unrelated sets of relationship attributes (watch statistics vs. rating/review content) at once.